THE UNIVERSITY OF
SYDNEY

ELEC5304

Project 1

Denoising Network

Student(s):
Dawod Ghifari 520140154
Hilal Chamtie 540749283
Darin Li 500048292
Akshar Hossain 540823996

Teacher:
Luping Zhou

April 17, 2026

Contents

1 Introduction **2**

2 Network Architecture **2**

3 Training Setup **3**

4 Results **4**

5 Analysis and Improvements **5**
5.1 Initial Model Limitations 5
5.2 Model and Training Improvements 5
5.3 Challenges and Generalisation 5

6 Additional Question: Speckle Noise **6**
6.1 Speckle Noise vs. Gaussian Noise 6
6.2 Choice of Speckle Noise Levels 6
6.3 PSNR Comparison 6

7 Conclusion **7**

1 Introduction

Image denoising is the process of recovering a clean image from a noisy or corrupted input. Noise can arise from many different sources, including shot noise — an unavoidable phenomenon in optical systems caused by the random, discrete arrival of photons (https://www.rp-photonics.com/shot_noise.html) — as well as sensor read noise in medical imaging, transmission errors in communications, and compression artefacts in video streaming. Since downstream tasks such as object detection, segmentation, and diagnosis all depend on image quality, denoising is an integral part of the pre-processing pipeline in computer vision and medical imaging systems. A good denoising algorithm suppresses noise while preserving fine image details such as edges, textures, and thin lines. Simple filtering approaches such as Gaussian blur fail this criterion, as they attenuate both the signal and the noise equally.

Utilising deep learning has fundamentally changed how this balance can be achieved. A Convolutional Neural Network (CNN) can learn noise-removal patterns from large image datasets far more effectively than traditional methods. In this project, a CNN image denoising model was implemented, inspired by the DnCNN architecture [5]. Rather than directly predicting the clean image, the network is trained to estimate the noise present in the input, which is then subtracted to recover the clean image. This residual learning formulation simplifies the learning task and yields more stable training.

This report describes the full pipeline from dataset preparation through network design, training, and evaluation. Section 2 covers the network architecture and key design choices. Section 3 covers the training setup, including data augmentation, the loss function, and the learning rate schedule. Section 4 presents the model's results, including training loss curves, PSNR curves, and visual comparisons of noisy, original, and restored images. Section 5 discusses what worked well, what did not, and why the final model outperforms simpler approaches. Section 6 examines whether a model trained on Gaussian noise can generalise to speckle noise.

2 Network Architecture

DnCNN (Denoising Convolutional Neural Network) is an architecture designed for image denoising that operates by stacking multiple convolutional layers sequentially, where each layer progressively captures more complex noise patterns. The same architecture is adopted here, with 17 convolutional layers producing a predicted noise map that is subtracted from the noisy input to recover a clean image.

The motivation for predicting noise rather than the clean image directly is that noise follows a consistent statistical pattern regardless of image content, making it a simpler and more stable learning target. The clean image, by contrast, contains rich and varied content that is substantially harder for the network to reconstruct from scratch. This formulation is known as residual learning: during training, the network processes noisy–clean image pairs, predicts the noise component, compares that prediction against the true noise, and adjusts its weights accordingly. After sufficient iterations, the network generalises to unseen images.

Several important design decisions shape the network:

- **Number of layers:** Too few layers limit the receptive field, causing the network to miss spatially extended noise patterns. Too many layers risk overfitting, where the network memorises training data rather than learning generalisable features. A depth of 17 layers balances these concerns.

- **Number of feature channels:** The network expands the single-channel greyscale input to 64 feature channels, providing sufficient representational capacity to distinguish noise from image content.
- **Batch normalisation:** Batch normalisation is applied after each intermediate convolutional layer. It standardises the activations passed between layers, preventing training instability caused by internal covariate shift and accelerating convergence.
- **ReLU activation:** Rectified Linear Unit (ReLU) activations are applied after each batch normalisation step. By setting negative activations to zero, ReLU introduces non-linearity, enabling the network to model complex, non-linear relationships between noisy and clean images.
- **Weight initialisation:** Convolutional filters are initialised as orthogonal matrices, ensuring that all filters are mutually independent at the start of training. This prevents vanishing or exploding activations and produces a neutral, well-conditioned starting point.

3 Training Setup

Of the 400 available images, 350 were allocated for training and 50 for testing. This split provides sufficient data for the network to learn from while retaining a held-out set for unbiased evaluation. Each greyscale image is 180×180 pixels, represented as a single-channel tensor with pixel values normalised to $[0, 1]$.

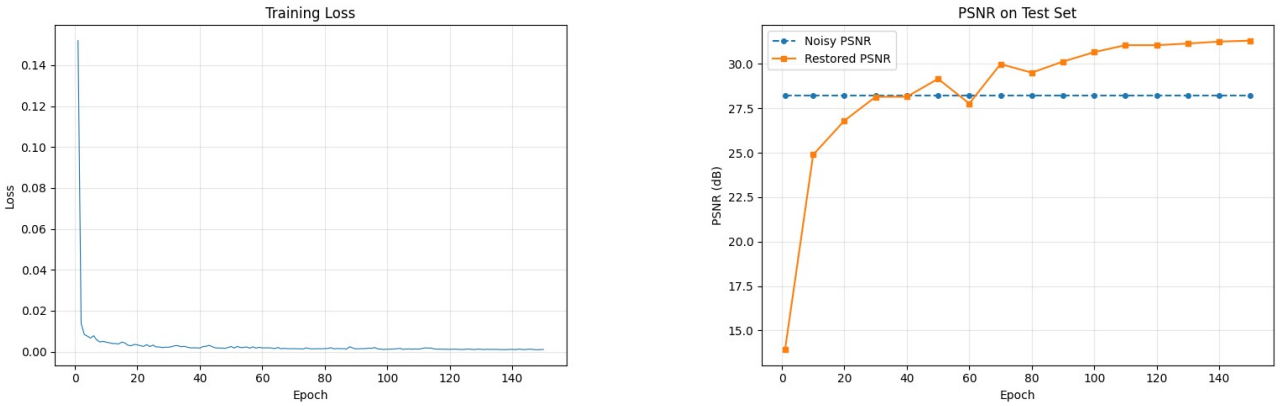
Prior to training, Gaussian noise was added to each image to construct noisy–clean pairs. Since only clean images are available, noisy versions must be synthesised artificially. Gaussian noise is a well-studied, mathematically tractable noise model that follows a normal distribution, making it a natural choice for training a residual denoiser. A noise standard deviation of $\sigma = 10$ (i.e., $10/255$ on the normalised scale) was applied. Noise is generated fresh for each training sample at each epoch, providing implicit data diversity. For the test set, noisy images were generated once with a fixed random seed (`torch.manual_seed(42)`) to ensure reproducible evaluation.

Data augmentation was applied during training to improve generalisation. Each training image was randomly flipped horizontally and vertically, each with a probability of 0.5. No augmentation was applied to the test set.

The network was trained for 150 epochs using Mean Squared Error (MSE) as the loss function. MSE penalises large prediction errors quadratically, encouraging the network to be conservative and avoid gross mistakes. The Adam optimiser was used to update weights after each mini-batch, with an initial learning rate of 0.001. Adam was preferred over SGD because it adapts the step size for each parameter based on the magnitude and consistency of past gradients, leading to faster and more stable convergence. A `CosineAnnealingLR` scheduler reduced the learning rate smoothly from 0.001 to 10^{-6} over the full training run, allowing large updates early in training and fine-grained adjustments near convergence. Gradient clipping with a maximum norm of 1.0 was also applied after each backward pass to prevent unstable gradient updates. PSNR on the test set was evaluated every 10 epochs to monitor progress.

4 Results

The network was trained using 150 epochs on 350 greyscale images, each of size 180×180 pixels and corrupted with additive Gaussian noise at $\sigma = 10$ (i.e., 10/255 on the $[0, 1]$ scale). Performance was evaluated on a held-out test set of 50 images using Peak Signal-to-Noise Ratio (PSNR), reported in decibels (dB), where higher values indicate better image quality. The PSNR of the noisy input serves as the baseline for comparison.



(a) Training Loss Curve

(b) PSNR Curve — Noisy Input vs. Restored

Figure 1: Training Loss Curve and PSNR Curve — Noisy Input vs. Restored

Figure 1a shows the MSE loss over training. The loss drops sharply in the first five epochs, then continues to decrease more gradually before flattening as the model converges. The curve is smooth with no sudden jumps, indicating stable training aided by gradient clipping and the cosine annealing schedule. No signs of overfitting are apparent, suggesting that the data augmentation was sufficient for this dataset size.

Figure 1b shows two PSNR curves evaluated every 10 epochs on the test set: (i) the PSNR of the noisy input relative to the clean reference, which remains constant at approximately 28.21 dB and serves as the baseline; and (ii) the PSNR of the network's restored output, which increases rapidly in early training and continues to improve gradually thereafter. The growing gap between the two curves confirms that the network is successfully learning to suppress Gaussian noise. By epoch 150, the restored PSNR reaches 31.29 dB, an improvement of approximately 3.1 dB over the noisy baseline.

Figure 2: Noisy / Original / Restored Side-by-Side

Figure 2 shows a side-by-side comparison of the noisy input, the clean reference, and the network output for a test example. The noisy image exhibits the typical uniform graininess of additive Gaussian noise. The restored image closely resembles the original, with edges largely preserved and smooth regions appearing noticeably cleaner. While a small amount of residual noise remains, it is substantially reduced relative to the noisy input. The absence of noticeable blurring confirms that the residual learning approach effectively separates noise from image structure.

5 Analysis and Improvements

5.1 Initial Model Limitations

The initial model used a shallow CNN with only four convolutional layers. After training for 30 epochs, the restored PSNR was approximately 26 dB — below the noisy input baseline of 28.21 dB — indicating that the model had not yet learned to denoise effectively. This limited performance is consistent with prior findings that shallow architectures lack the capacity to model complex noise distributions [1, 2]. Additionally, the initial learning rate schedule (StepLR) reduced the learning rate in discrete steps, producing relatively abrupt transitions that made the optimisation process less smooth.

5.2 Model and Training Improvements

To improve performance, the network was deepened to 17 layers and training was extended to 150 epochs. Increased depth is well established as a means of improving feature extraction and denoising performance in CNNs [2]. Concurrently, the learning rate schedule was changed from StepLR to Cosine Annealing, which decays the learning rate continuously and smoothly. Unlike StepLR, which applies sudden reductions at fixed intervals, Cosine Annealing allows the model to learn quickly in the early stages and make increasingly fine-grained adjustments as training progresses. This smoother scheduling strategy is widely adopted in modern deep learning and has been shown to improve convergence stability [3]. Together, these changes brought the final restored PSNR to 31.29 dB, an improvement of 3.1 dB over the noisy baseline.

5.3 Challenges and Generalisation

Several challenges were encountered during development. The initial shallow model produced output images with noticeable residual noise, a symptom of underfitting consistent with prior work on the limitations of shallow denoising architectures [1]. Ensuring stable training required careful tuning of the learning rate schedule and gradient clipping.

The model's performance was also observed to depend strongly on the type of noise encountered. While the network performs well on Gaussian noise — the distribution it was trained on — its performance on speckle noise was less consistent. This is consistent with findings in the literature showing that models trained on a specific noise distribution often struggle to generalise to structurally different noise types [4]. Detailed results are presented in Section 6.

6 Additional Question: Speckle Noise

6.1 Speckle Noise vs. Gaussian Noise

Gaussian noise is additive and signal-independent: a noise term drawn from a normal distribution is simply added to the clean image pixel values. Speckle noise, by contrast, is multiplicative and signal-dependent. In this implementation, speckle noise is applied by scaling each pixel by a random factor:

$$x_{\text{noisy}} = x_{\text{clean}} \cdot (1 + \sigma\epsilon), \quad \epsilon \sim \mathcal{N}(0, 1),$$

where ϵ is sampled independently for each pixel. Because the noise magnitude scales with the underlying pixel intensity, bright regions are corrupted more severely than dark ones, producing fundamentally different visual and statistical properties compared to additive Gaussian noise.

6.2 Choice of Speckle Noise Levels

Three speckle noise levels were selected: $\sigma = 0.04$, $\sigma = 0.06$, and $\sigma = 0.10$. These values were chosen by comparing the perceptual degradation of speckle-corrupted images against that of Gaussian-corrupted images at $\sigma = 10$. The noisy PSNR values confirm the correspondence: $\sigma = 0.06$ (noisy PSNR 30.57 dB) and $\sigma = 0.10$ (noisy PSNR 26.23 dB) bracket the Gaussian baseline of 28.21 dB most closely. The level $\sigma = 0.04$ (noisy PSNR 34.04 dB) produces substantially less corruption than the Gaussian training noise, which is reflected in the denoising results below.

6.3 PSNR Comparison

The trained model was evaluated on both Gaussian- and speckle-corrupted test sets. PSNR was measured before and after denoising, and the improvement (PSNR After − PSNR Before) was computed. Results are summarised in Table 1.

Table 1: PSNR Comparison: Gaussian vs. Speckle Noise

Noise Type	**PSNR Before (dB)**	**PSNR After (dB)**	**Improvement (dB)**
Gaussian ($\sigma = 10$)	28.21	30.32	+2.10
Speckle ($\sigma = 0.04$)	34.04	31.22	−2.82
Speckle ($\sigma = 0.06$)	30.57	30.78	+0.21
Speckle ($\sigma = 0.10$)	26.23	28.19	+1.96

The model achieves its strongest improvement on Gaussian noise (+2.10 dB), consistent with it being the training distribution. For speckle noise, results are more nuanced. At $\sigma = 0.10$, where the corruption level is closest to the training noise, the model achieves a comparable improvement of +1.96 dB. At $\sigma = 0.06$, the improvement is marginal (+0.21 dB). Notably, at $\sigma = 0.04$, the model *degrades* image quality by 2.82 dB: because the input is already of high quality (noisy PSNR 34.04 dB), the network over-processes it by applying corrections calibrated for heavier Gaussian-level corruption. This result highlights that the model has implicitly learned to assume a specific noise level, and its performance deteriorates when that assumption is violated.

7 Conclusion

A model trained on Gaussian noise can partially generalise to speckle noise, particularly when the magnitude of corruption is similar to that of the training noise. However, due to the fundamental difference between additive (Gaussian) and multiplicative (speckle) noise, performance varies considerably across speckle noise levels. At noise levels close to the training distribution ($\sigma = 0.10$), the model achieves meaningful improvement. At lower noise levels ($\sigma = 0.04$), the model over-processes the image and reduces its quality. These results indicate that while the network has acquired general image restoration capabilities, its performance is closely tied to the noise distribution and magnitude seen during training.

References

[1] A. E. Ilesanmi and T. O. Ilesanmi, "Methods for image denoising using convolutional neural network: A review," *Complex & Intelligent Systems*, vol. 7, no. 2, pp. 1–25, 2021.

[2] C. Tian, Y. Xu, L. Fei, and K. Yan, "Deep learning for image denoising: A survey," *arXiv preprint arXiv:1810.05052*, 2018.

[3] A. Saini and A. Dogra, "Unleashing the power of image denoising: A comprehensive review of classical to deep learning methods," *Journal of Computer Science*, 2025.

[4] R. S. Jebur *et al.*, "A comprehensive review of image denoising in deep learning," *Multimedia Tools and Applications*, 2023.

[5] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, Jul. 2017.